

Java Memory Management

A Complete Guide to Heap Structure, Garbage Collection Algorithms & JVM Tuning

1. Java Memory Management Basics

The Two Main Areas of Memory

Java divides memory into the **heap** and the **stack**. The stack holds local variables and method calls — each entry is removed as soon as a method finishes. It is fast, organized, and automatic. The heap is where objects (anything created with `new`) live. Objects remain there until no reference points to them.

How References Work

When you write `Dog myDog = new Dog()`, two things happen. The actual `Dog` object is stored on the heap, and `myDog` is a reference sitting on the stack that points to it. Multiple references can point to the same object — like multiple people having the same warehouse address.

Garbage Collection — The Automatic Cleanup Crew

Java has a **Garbage Collector (GC)** that periodically scans the heap and removes objects with no remaining references. You do not have to manually free memory like in C/C++. You can suggest garbage collection with `System.gc()`, but the JVM is free to ignore you.

Memory leaks can still happen if you accidentally hold onto references you no longer need — for example, adding objects to a list and never removing them.

2. Heap Structure — Generations

The key insight behind Java's GC design is that **most objects die young**. A temporary string, a loop variable, a short-lived calculation — they are created, used once, and forgotten. Java exploits this by splitting the heap into generations.

Young Generation

The Young Generation is where new objects are born. It is further split into three parts:

Eden Space — Every new object starts here. Think of it as a nursery. When you write `new Dog()`, that `Dog` lands in Eden. Eden fills up fast because objects are constantly being created.

Survivor Space 0 and Survivor Space 1 (S0 & S1) — Two small holding areas. Only one is active at a time; the other is always empty. They act as a waiting room for objects that survived at least one GC cycle but have not proven they are long-lived enough for the Old Generation.

How a Minor GC Cycle Works

1. Your program creates objects, filling up Eden.
2. Eden gets full — this triggers a **Minor GC**.
3. The GC checks every object in Eden. Anything still referenced gets copied to the active Survivor space (say S0). Everything else is wiped.
4. Next time Eden fills up, surviving objects from both Eden and S0 get copied to S1. Then Eden and S0 are wiped clean.
5. The two Survivor spaces keep swapping roles. Each time an object survives, its **age counter** goes up.
6. Once an object's age hits a threshold (default ~15), it is **promoted** to the Old Generation.

Old Generation

Objects that have survived many GC cycles live here. This space is bigger and collected less frequently. When it fills up, a **Major GC (Full GC)** runs, which is much slower and can cause noticeable pauses — the dreaded stop-the-world event.

Why This Design Is Clever

Since most objects die almost immediately, Minor GC only needs to copy the few survivors. Clearing Eden is essentially free because it just resets a pointer. Minor GCs are extremely fast (usually milliseconds). The expensive Full GC rarely runs because most garbage never makes it to the Old Generation.

Analogy: Eden is the intern pool — lots of people come in, most leave quickly. Survivor spaces are short-term contracts. The Old Generation is permanent staff. HR (the GC) clears out interns constantly (cheap) but only does a full company restructuring (Full GC) when absolutely necessary.

3. Garbage Collection Algorithms

Serial GC

Uses a single thread for all garbage collection. When it runs, everything stops (stop-the-world). Fine for small apps or command-line tools with small heaps. Enable with `-XX:+UseSerialGC`.

Parallel GC (Throughput Collector)

Same idea as Serial, but uses **multiple threads** for Minor GC. Still causes stop-the-world pauses, but shorter ones. Prioritizes throughput — maximizing time spent doing actual work vs. GC. Default in Java 8. Enable with `-XX:+UseParallelGC`.

CMS (Concurrent Mark-Sweep)

The first attempt at reducing pause times. Does most work **concurrently** while the application runs. Only pauses briefly for initial mark and remark phases. Downside: no compaction, so the Old Generation fragments over time, eventually forcing a slow Full GC. Deprecated in Java 9, removed in Java 14.

G1 (Garbage First) — The Modern Default

Default since Java 9. Instead of strict Young/Old regions, G1 splits the entire heap into many small **regions** (typically 1–32 MB each). Each region can be Eden, Survivor, or Old dynamically. G1 tracks garbage per region and collects the regions with the most garbage first (hence the name). It copies live objects to fresh regions, freeing dirty ones entirely.

G1's key feature is the **pause time goal**. You tell it your target with `-XX:MaxGCPauseMillis=200`, and it adjusts how many regions it collects per cycle to meet that target.

ZGC — The Low-Latency Beast

Introduced in Java 11, production-ready in Java 15. Designed for **ultra-low pause times** regardless of heap size. Can handle multi-terabyte heaps with pauses under 1–2 milliseconds. Uses **colored pointers** — metadata stored in unused bits of 64-bit object pointers. Almost all work happens concurrently. Tradeoff: slightly lower throughput and higher CPU usage.

Shenandoah

Similar goals to ZGC — sub-millisecond pauses — but uses concurrent compaction with Brooks pointers (an extra forwarding pointer per object). Developed by Red Hat, available in OpenJDK but not Oracle JDK.

GC Algorithm Comparison

Algorithm	Pauses	Throughput	Best For
Serial	Long	Low	Small apps, single-core

Parallel	Medium	High	Batch jobs, throughput
CMS	Short	Medium	Deprecated (Java 14)
G1	Configurable	Good	General purpose (default)
ZGC	<2ms	Good	Latency-sensitive apps
Shenandoah	<1ms	Good	Latency-sensitive (OpenJDK)

4. JVM Tuning Flags

Heap Size — The Fundamentals

-Xms512m sets the initial heap size. -Xmx2g sets the maximum. A common practice is to set both to the same value so the JVM does not waste time resizing at runtime. Too small and you get frequent GC or OutOfMemoryError. Too big and GC pauses get longer.

Young Generation Sizing

-XX:NewRatio=2 means Old Gen is 2x the size of Young Gen (Young Gen gets 1/3 of heap).
-XX:NewSize and -XX:MaxNewSize let you set it directly. Larger Young Gen = fewer but slightly longer Minor GCs. Smaller = more frequent but faster Minor GCs.

Survivor Space Tuning

-XX:SurvivorRatio=8 means Eden is 8x each Survivor space (8:1:1 split).
-XX:MaxTenuringThreshold=15 controls how many GC cycles before promotion to Old Gen. Lower this if objects are clearly long-lived and you are wasting time copying between Survivor spaces.

G1-Specific Flags

Flag	Purpose	Default
-XX:MaxGCPauseMillis	Pause time target	200ms
-XX:G1HeapRegionSize	Region size (power of 2)	1-32 MB (auto)
-XX:InitiatingHeapOccupancyPercent	Start concurrent cycle at this heap %	45

ZGC-Specific Flags

-XX:+UseZGC enables ZGC. -XX:SoftMaxHeapSize=4g with -Xmx8g tells ZGC to try to stay under 4 GB but allows up to 8 GB if needed. ZGC needs very little tuning by design.

Diagnostic Flags

Flag	Purpose
-Xlog:gc*	Enable GC logging (Java 9+)
-XX:+HeapDumpOnOutOfMemoryError	Create heap dump on OOM crash
-XX:HeapDumpPath=/path	Where to save heap dumps

Practical Starting Point for a Web App

```
java -Xms2g -Xmx2g -XX:+UseG1GC -XX:MaxGCPauseMillis=200 -Xlog:gc*:file=gc.log  
-XX:+HeapDumpOnOutOfMemoryError -jar myapp.jar
```

This gives you a 2 GB heap, G1 collector with a 200ms pause target, GC logging for analysis, and a heap dump if things go wrong. Start with defaults, measure with GC logs, change one flag at a time, and measure again.